

Name - Amit Kumar

Course - HPC

Day 1: Question bank

Q.No.	Correct Answer
1	B – A device that processes data into information
2	B – A set of instructions for the computer
3	B – To communicate with other humans
4	B – C
5	B – The rules for writing code in a language
6	B – A step-by-step procedure to solve a problem
7	C – Machine language
8	B – Written using mnemonics
9	B – Assembler
10	A – Compiler
11	B – Interpreter
12	D – Executes faster than machine language
13	C – Binary codes (0s and 1s)
14	B – HTML
15	C – To convert human-readable code into machine-readable form
16	B – They generate a separate executable file
17	B – Python
18	C – Syntax error
19	C – %d
20	D – total marks

Lab Assignment: Practical C Programming

1. Create a program to declare two integer variables, assign them values, and display their values

```
C test4.c > main()
1  #include<stdio.h>
2
3  int main()
4  {
5      int a = 10, b = 20;
6
7      printf("Value of a = %d\n", a);
8      printf("Value of b = %d\n", b);
9
10     return 0;
11 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
```

```
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
```

```
Value of a = 10
```

```
Value of b = 20
```

```
PS C:\Users\STUDENT\Desktop\Amit\C> 
```

2. Create a program to input the name of the person and respond with "Welcome NAME to C Programming".

```
test4.c > main()
1  #include <stdio.h>
2
3  int main() {
4      char name[50];
5
6      printf("Enter your name: ");
7      scanf("%s", name);
8
9      printf("Welcome %s to C Programming\n", name);
10
11     return 0;
12 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4

PS C:\Users\STUDENT\Desktop\Amit\C> ./test4

Enter your name: Amit Kumar

Welcome Amit to C Programming

PS C:\Users\STUDENT\Desktop\Amit\C> █

3. Define variables for storing a user's first name, last name, and age using appropriate naming conventions and then display them.

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main() {
4      char firstName[30], lastName[30];
5      int age;
6
7      printf("Enter First Name: ");
8      scanf("%s", firstName);
9
10     printf("Enter Last Name: ");
11     scanf("%s", lastName);
12
13     printf("Enter Age: ");
14     scanf("%d", &age);
15
16     printf("\nFirst Name : %s", firstName);
17     printf("\nLast Name  : %s", lastName);
18     printf("\nAge       : %d", age);
19
20     return 0;
21 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
```

```
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
```

```
Enter First Name: Amit
```

```
Enter Last Name: Kumar
```

```
Enter Age: 23
```

```
First Name : Amit
```

```
Last Name  : Kumar
```

```
Age       : 23
```

```
PS C:\Users\STUDENT\Desktop\Amit\C> █
```

4. Create a program to print the area of a square by inputting its side length.

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main() {
4      float side, area;
5
6      printf("Enter side of square: ");
7      scanf("%f", &side);
8
9      area = side * side;
10
11     printf("Area of Square = %.2f\n", area);
12
13     return 0;
14 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
```

```
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
```

```
Enter side of square: 5
```

```
Area of Square = 25.00
```

```
PS C:\Users\STUDENT\Desktop\Amit\C> █
```

5. Create a program to define a constant for the mathematical value pi (3.14159) and use it to calculate and print the circumference of a circle with a radius input from the user.

```
C test4.c > main()
1  #include <stdio.h>
2
3  #define PI 3.14159
4
5  int main() {
6      float radius, circumference;
7
8      printf("Enter radius: ");
9      scanf("%f", &radius);
10
11     circumference = 2 * PI * radius;
12
13     printf("Circumference = %.2f\n", circumference);
14
15     return 0;
16 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
Enter radius: 5
Circumference = 31.42
PS C:\Users\STUDENT\Desktop\Amit\C> 
```

6. Create a program to print the area of a circle by inputting its radius.

```
C test4.c > main()
1  #include <stdio.h>
2
3  #define PI 3.14159
4
5  int main() {
6      float radius, area;
7
8      printf("Enter radius: ");
9      scanf("%f", &radius);
10
11     area = PI * radius * radius;
12
13     printf("Area of Circle = %.2f\n", area);
14
15     return 0;
16 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
Enter radius: 5
Area of Circle = 78.54
PS C:\Users\STUDENT\Desktop\Amit\C> █
```

7. Create a program to swap two numbers.

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int a, b, temp;
5
6      printf("Enter two numbers: ");
7      scanf("%d%d", &a, &b);
8
9      temp = a;
10     a = b;
11     b = temp;
12
13     printf("After Swapping:\n");
14     printf("a = %d\n", a);
15     printf("b = %d\n", b);
16
17     return 0;
18 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
```

```
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
```

```
Enter two numbers: 36
```

```
63
```

```
After Swapping:
```

```
a = 63
```

```
b = 36
```

```
PS C:\Users\STUDENT\Desktop\Amit\C> █
```


8. Write a C program that asks the user to input two integers, then performs and displays the results of the following arithmetic operations: Addition Subtraction Multiplication Division (show the result as a float) Modulus (remainder) Make sure the program properly handles integer inputs and prints the results clearly.  Sample Output: Enter two integers: 10 3 Results: Addition = 13 Subtraction = 7 Multiplication = 30 Division = 3.33 Modulus = 1

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main() {
4      int a, b;
5
6      printf("Enter two integers: ");
7      scanf("%d%d", &a, &b);
8
9      printf("\nResults:\n");
10     printf("Addition = %d\n", a + b);
11     printf("Subtraction = %d\n", a - b);
12     printf("Multiplication = %d\n", a * b);
13     printf("Division = %.2f\n", (float)a / b);
14     printf("Modulus = %d\n", a % b);
15
16     return 0;
17 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
Enter two integers: 8
9

Results:
Addition = 17
Subtraction = -1
Multiplication = 72
Division = 0.89
Modulus = 8
PS C:\Users\STUDENT\Desktop\Amit\C> |
```

9. Write a program to calculate Simple Interest using the formula $\text{interest} = (\text{principal} * \text{rate} * \text{time}) / 100$

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main() {
4      float principal, rate, time, interest;
5
6      printf("Enter Principal: ");
7      scanf("%f", &principal);
8
9      printf("Enter Rate: ");
10     scanf("%f", &rate);
11
12     printf("Enter Time: ");
13     scanf("%f", &time);
14
15     interest = (principal * rate * time) / 100;
16
17     printf("Simple Interest = %.2f\n", interest);
18
19     return 0;
20 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
Enter Principal: 100
Enter Rate: 10
Enter Time: 3
Simple Interest = 30.00
PS C:\Users\STUDENT\Desktop\Amit\C> 
```

10. Write a program to add, subtract, multiply, and divide two numbers.

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main() {
4      float num1, num2;
5
6      printf("Enter two numbers: ");
7      scanf("%f%f", &num1, &num2);
8
9      printf("Addition = %.2f\n", num1 + num2);
10     printf("Subtraction = %.2f\n", num1 - num2);
11     printf("Multiplication = %.2f\n", num1 * num2);
12
13     if (num2 != 0)
14         printf("Division = %.2f\n", num1 / num2);
15     else
16         printf("Division not possible (divide by zero)\n");
17
18     return 0;
19 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
```

```
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
```

```
Enter two numbers: 20 10
```

```
Addition = 30.00
```

```
Subtraction = 10.00
```

```
Multiplication = 200.00
```

```
Division = 2.00
```

```
PS C:\Users\STUDENT\Desktop\Amit\C> █
```

11. Write a program to calculate Compound Interest. $\text{amount} = p * \text{pow}((1 + r / 100), t)$ $\text{ci} = \text{amount} - p$; Hint: Use #include

```
C test4.c > main()
1  #include <stdio.h>
2  #include <math.h>
3
4  int main()
5  {
6      float p, r, t, amount, ci;
7
8      printf("Enter Principal: ");
9      scanf("%f", &p);
10
11     printf("Enter Rate: ");
12     scanf("%f", &r);
13
14     printf("Enter Time: ");
15     scanf("%f", &t);
16
17     amount = p * pow((1 + r / 100), t);
18     ci = amount - p;
19
20     printf("Amount = %.2f\n", amount);
21     printf("Compound Interest = %.2f\n", ci);
22
23     return 0;
24 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
Enter Principal: 100
Enter Rate: 5
Enter Time: 2
Amount = 110.25
Compound Interest = 10.25
PS C:\Users\STUDENT\Desktop\Amit\C> █
```

12. Write a program to calculate BMI (Body Mass Index).
$$\text{bmi} = \text{weight} / (\text{height} * \text{height})$$

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      float weight, height, bmi;
6
7      printf("Enter Weight (kg): ");
8      scanf("%f", &weight);
9
10     printf("Enter Height (m): ");
11     scanf("%f", &height);
12
13     bmi = weight / (height * height);
14
15     printf("BMI = %.2f\n", bmi);
16
17     return 0;
18 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
```

```
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
```

```
Enter Weight (kg): 45
```

```
Enter Height (m): 2.5
```

```
BMI = 7.20
```

```
PS C:\Users\STUDENT\Desktop\Amit\C> 
```

13. Write a program to convert Celsius to Fahrenheit. $f = (c * 9 / 5) + 32$

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      float c, f;
6
7      printf("Enter Temperature in Celsius: ");
8      scanf("%f", &c);
9
10     f = (c * 9 / 5) + 32;
11
12     printf("Temperature in Fahrenheit = %.2f\n", f);
13
14     return 0;
15 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
Enter Temperature in Celsius: 30
Temperature in Fahrenheit = 86.00
PS C:\Users\STUDENT\Desktop\Amit\C> █
```

14. Write a program to convert Fahrenheit to Celsius. $c = (f - 32) * 5 / 9$

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      float f, c;
6
7      printf("Enter Temperature in Fahrenheit: ");
8      scanf("%f", &f);
9
10     c = (f - 32) * 5 / 9;
11
12     printf("Temperature in Celsius = %.2f\n", c);
13
14     return 0;
15 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
Enter Temperature in Fahrenheit: 100
Temperature in Celsius = 37.78
PS C:\Users\STUDENT\Desktop\Amit\C> 
```

15. Write a program to calculate the percentage of marks (5 subjects, out of 100 each). $\text{total} = m1 + m2 + m3 + m4 + m5$ $\text{percentage} = \text{total} / 5$

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      float m1, m2, m3, m4, m5;
6      float total, percentage;
7
8      printf("Enter Marks of 5 Subjects:\n");
9      scanf("%f%f%f%f%f", &m1, &m2, &m3, &m4, &m5);
10
11     total = m1 + m2 + m3 + m4 + m5;
12     percentage = total / 5;
13
14     printf("Total Marks = %.2f\n", total);
15     printf("Percentage = %.2f%%\n", percentage);
16
17     return 0;
18 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
Enter Marks of 5 Subjects:
93
90
86
82
85
Total Marks = 436.00
Percentage = 87.20%
PS C:\Users\STUDENT\Desktop\Amit\C> █
```


16. Write a program to calculate the discounted price of an item. $\text{discount} = (\text{price} * \text{discountPercent}) / 100$ $\text{finalPrice} = \text{price} - \text{discount}$

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      float price, discountPercent;
6      float discount, finalPrice;
7
8      printf("Enter Price: ");
9      scanf("%f", &price);
10
11     printf("Enter Discount Percentage: ");
12     scanf("%f", &discountPercent);
13
14     discount = (price * discountPercent) / 100;
15     finalPrice = price - discount;
16
17     printf("Discount = %.2f\n", discount);
18     printf("Final Price = %.2f\n", finalPrice);
19
20     return 0;
21 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
Enter Price: 100
Enter Discount Percentage: 10
Discount = 10.00
Final Price = 90.00
PS C:\Users\STUDENT\Desktop\Amit\C> █
```

17. Write a program to calculate speed, given distance and time. $\text{speed} = \text{distance} / \text{time}$

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      float distance, time, speed;
6
7      printf("Enter Distance: ");
8      scanf("%f", &distance);
9
10     printf("Enter Time: ");
11     scanf("%f", &time);
12
13     speed = distance / time;
14
15     printf("Speed = %.2f\n", speed);
16
17     return 0;
18 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
Enter Distance: 100
Enter Time: 10
Speed = 10.00
PS C:\Users\STUDENT\Desktop\Amit\C> █
```

18. Write a program to calculate net salary = (basic pay + allowance – deductions).

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      float basic, allowance, deduction, salary;
6
7      printf("Enter Basic Pay: ");
8      scanf("%f", &basic);
9
10     printf("Enter Allowance: ");
11     scanf("%f", &allowance);
12
13     printf("Enter Deduction: ");
14     scanf("%f", &deduction);
15
16     salary = basic + allowance - deduction;
17
18     printf("Net Salary = %.2f\n", salary);
19
20     return 0;
21 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
Enter Basic Pay: 8000
Enter Allowance: 2000
Enter Deduction: 1000
Net Salary = 9000.00
PS C:\Users\STUDENT\Desktop\Amit\C> |
```

19. Predict the output and verify it by running the program. #include int main() { int x = 5; printf("x = %d\n", x); printf("x++ = %d\n", x++); printf("After x++ = %d\n", x); printf("++x = %d\n", ++x); printf("Final x = %d\n", x); return 0; }

C test4.c > main()

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int x = 5;
6
7      printf("x = %d\n", x);
8      printf("x++ = %d\n", x++);
9      printf("After x++ = %d\n", x);
10     printf("++x = %d\n", ++x);
11     printf("Final x = %d\n", x);
12
13     return 0;
14 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4

PS C:\Users\STUDENT\Desktop\Amit\C> ./test4

x = 5

x++ = 5

After x++ = 6

++x = 7

Final x = 7

PS C:\Users\STUDENT\Desktop\Amit\C> █

20. Predict the output and verify it by running the program. #include int main() { int i = 6, j = 3, k; k = ++i * j--; printf("i = %d\n", i); printf("j = %d\n", j); printf("k = %d\n", k); return 0; }

```
C test4.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      int i = 6, j = 3, k;
6
7      k = ++i * j--;
8
9      printf("i = %d\n", i);
10     printf("j = %d\n", j);
11     printf("k = %d\n", k);
12
13     return 0;
14 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\STUDENT\Desktop\Amit\C> gcc test4.c -o test4
PS C:\Users\STUDENT\Desktop\Amit\C> ./test4
i = 7
j = 2
k = 21
PS C:\Users\STUDENT\Desktop\Amit\C> █
```

Explain:

1. Why is C called a mid-level programming language?

C is called a mid-level programming language because it combines the features of both low-level and high-level languages.

Supports low-level programming using pointers and direct memory access.

Supports high-level programming using functions, loops, and structures.

Used for both system software and application software.

2. What are the features of the C programming language?

Features of C:

Simple and easy to learn

Portable (runs on different platforms)

Fast execution

Structured programming

Rich library functions

Supports pointers

Dynamic memory allocation

Modular programming using functions

3. Explain the internal execution of a program to swap two numbers.

Steps:

Declare two variables (a and b) and a temporary variable (temp).

Store the value of a in temp.

Copy the value of b into a.

Copy the value of temp into b.

Print the swapped values.

Example:

Before:

a = 10

b = 20

After:

a = 20

b = 10

4. When is your program ready to execute?

A C program is ready to execute when:

The source code has no syntax errors.

It is successfully compiled.

The linker creates an executable file (. exe in Windows).

5. Explain the compilation process in C.

The compilation process consists of four stages:

Preprocessing – Processes directives like #include and #define.

Compilation – Converts C code into assembly code.

Assembly – Converts assembly code into object code.

Linking – Combines object code with library files to create the executable program.

6. Difference between Compiler and Interpreter

Compiler	Interpreter
Translates the whole program at once	Translates one line at a time
Produces an executable file	Does not produce an executable file
Faster execution	Slower execution
Errors shown after compilation	Errors shown line by line
Example: GCC	Example: Python Interpreter

7. What is the purpose of `#include <stdio.h>` in a C program?

`#include <stdio.h>` includes the Standard Input/Output Header File.

It provides declarations for functions such as:

- `printf()`
- `scanf()`

Without this header, these functions cannot be used properly.

8. Explain the difference between `printf()` and `scanf()` with examples.

`printf()` : Used to display output on the screen.

Example:

```
printf("Hello World");
```

`scanf()` : Used to take input from the user.

Example:

```
int age;  
scanf("%d", &age);
```

<code>printf()</code>	<code>scanf()</code>
Output function	Input function
Displays data	Reads data
No & needed	& is required for variables (except character arrays)

9. What are format specifiers in C? Explain any five.

Format specifiers tell `printf()` and `scanf()` what type of data is being used.

Specifier	Data Type	Example
%d	Integer	<code>int a = 10;</code>
%f	Float	<code>float x = 3.5;</code>
%c	Character	<code>char ch = 'A';</code>
%s	String	<code>char name[] = "Amit";</code>
%lf	Double	<code>double d = 12.45;</code>

10. Explain the difference between pre-increment (`++i`) and post-increment (`i++`) with an example.

Pre-increment (`++i`): The value is incremented before it is used.

Example:

```
int i = 5;
printf("%d", ++i);
```

Output:6

Post-increment (`i++`): The current value is used first, then it is incremented.

Example:

```
int i = 5;
printf("%d", i++);
```

Output:5

After execution:

`i = 6`

11. Why do we use the `main()` function in every C program?

The `main()` function is the entry point of a C program.

When a C program starts executing, the operating system calls the `main()` function first.

12. What is the purpose of the `return 0;` statement?

`return 0;` is used to indicate that the `main()` function has completed successfully.

`return 0;` indicates successful completion of the C program.

13. Why do we use the address operator (`&`) in `scanf()`?

The address operator `&` gives the memory address of a variable.

`scanf()` needs the address of the variable so that it can store the user's input directly into that memory location.

Example:

```
int age;
```

```
scanf("%d", &age);
```

Here: `&age` means the address of `age`.

The value entered by the user is stored at that address.

Important:

Correct:

```
scanf("%d", &age);
```

Usually incorrect for a normal integer variable:

```
scanf("%d", age);
```

In simple words: & tells scanf () where to store the input value.

14. What is the difference between float and double?

Both float and double are used to store decimal/floating-point numbers.

Feature	float	double
Typical size	4 bytes	8 bytes
Precision	Lower	Higher
Approx. precision	6–7 digits	15–16 digits
Memory	Less	More
Example	3.14f	3.1415926535

Use float when normal precision is sufficient and double when higher precision is required.

15. Explain the difference between unary operators and arithmetic operators with examples.

Unary Operators : A unary operator works on only one operand.

Examples: ++i, --i, -i

Arithmetic Operators: Arithmetic operators are used to perform mathematical operations.

Common arithmetic operators are:

- + Addition
- Subtraction
- * Multiplication
- / Division
- % Modulus

Difference:

Unary Operator	Arithmetic Operator
Works on one operand	Usually works on two operands
++, --, unary -	+, -, *, /, %
Example: ++a	Example: a + b

16. What is a variable in C? Explain the rules for naming (identifiers) a variable with examples.

A variable is a named memory location used to store data whose value can change during program execution.

Example: `int age = 24;`

Here:

- `int` → data type
- `age` → variable name
- `24` → value

Rules for naming variables

1. Name can contain letters, digits and underscore.
2. Name cannot start with a digit.
3. Spaces are not allowed.
4. C keywords cannot be used as variable names.
5. C is case-sensitive.
6. Special characters such as `@`, `#`, `$` are not allowed.

Valid identifiers:

`age`
`student_name`
`marks1`
`_total`

Invalid identifiers:

1student // starts with digit
student name // contains space
float // keyword
student@name // special character

Example:

```
int student_age = 20;
```

17. What are literals in C? Differentiate between constants and literals with examples.

Literal : A literal is a fixed value written directly in the source code.

Examples: 10, 3.14, 'A', "Hello"

Example: `int age = 20;` // Here 20 is an integer literal.

Constant: A constant is a value that cannot be changed during program execution.

Example: `const int MAX = 100;` // Here MAX is a constant.

Difference:

Literal	Constant
Fixed value written directly in code	Named or defined value intended not to change
Example: 100	Example: <code>const int MAX = 100</code>
Usually has no name	Usually has an identifier
Can be used as a value	Represents a fixed value

18. What are keywords in C? Why can't keywords be used as variable names?

Keywords are reserved words that have a predefined meaning in the C language.

Examples: int, float, if, else, while, for, return, void, char, const

They cannot be used as variable names because the compiler already assigns a specific meaning to them.

Incorrect:

```
int int = 10;
```

This is invalid because `int` is a keyword.

Correct:

```
int number = 10;
```

Another example:

```
int return = 5; // Invalid
```

Because `return` is a keyword.

19. What is the purpose of the void keyword in C? Explain its different uses with examples.

The `void` keyword means "no value" or "no specific type."

It is mainly used in three ways.

1. Function returns no value

```
void display()  
{  
    printf("Hello");  
}
```

This function performs an operation but does not return a value.

2. Function accepts no arguments

```
void display(void)  
{  
    printf("Hello");  
}
```

Here void inside the parentheses indicates that the function takes no parameters.

3. Void pointer

A void * pointer can store the address of a variable of different data types.

Example:

```
int x = 10;  
void *ptr = &x;
```

Here ptr is a generic pointer.

20. Differentiate between variables, constants, and literals with suitable examples.

Variable: A variable is a named memory location whose value can change.

```
int age = 20;,  
age = 21;  
Here age is a variable.
```

Constant: A constant represents a value that should not change during program execution.

```
const int MAX_MARKS = 100;
```

MAX_MARKS cannot normally be assigned another value.

Literal: A literal is a fixed value written directly in the program.

Examples: 20, 100, 3.14, 'A', "Hello"

Comparison:

Feature	Variable	Constant	Literal
Meaning	Changeable value	Fixed value	Direct value written in code
Has name?	Yes	Usually yes	Usually no

Can change?	Yes	No	Itself is a fixed value
Example	age	MAX_MARKS	100
Declaration	<code>int age = 20;</code>	<code>const int MAX = 100;</code>	<code>int x = 100;</code>